

Lab 3 – Testing & CI/CD for ML Systems

Movie Rating Prediction API (FastAPI + SVD, MovieLens 100K)

Student	Vương Xuân Thông
Student ID	25MS13306
Repository	https://github.com/thongvx/ddm501-lab3-testing-cicd
CI status	https://github.com/thongvx/ddm501-lab3-testing-cicd/actions – all workflows passing
Release	v1.0.0 · image <code>ghcr.io/thongvx/movie-rating-api:1.0.0</code>
Submission date	October 2026

1. Deliverables vs. grading rubric

Criterion	Evidence	Status
Unit tests (10%)	63 tests: model wrapper (real + isolated stub model), schemas, config, helpers – <code>tests/unit/</code>	✓
Integration tests (8%)	44 tests: all endpoints, 404/405/422/500/503, batch, CORS, latency – <code>tests/integration/</code>	✓
Data tests (6%)	32 tests: sample fixture + real MovieLens 100K data contract – <code>tests/data/</code>	✓
Model behavioural tests (6%)	23 tests: invariance, directional, minimum functionality, performance, robustness – <code>tests/model/</code>	✓
CI workflow works (12%)	<code>ci.yml</code> : lint (pre-commit) · mypy → tests on Python 3.10 & 3.11 → Docker build + smoke test (Fig. 2–3)	✓
All checks pass (10%)	Every run green on <code>main</code> , <code>develop</code> and tag <code>v1.0.0</code> ; badges "passing" (Fig. 1–2)	✓
CD workflow configured (8%)	<code>cd.yml</code> : CI gate → push to GHCR → staging smoke test → GitHub Release → production; plus <code>model-validation.yml</code> (Fig. 4–6)	✓
Pre-commit hooks (8%)	hygiene hooks, black, isort, flake8, mypy, local pytest hook; same hooks run in CI	✓
Linting passes (6%)	black, isort, flake8 – 0 issues (CI job "Lint")	✓
Type hints (6%)	all of <code>app/</code> and <code>scripts/</code> typed; mypy <code>--disallow-untyped-defs</code> 0 errors	✓
Testing strategy doc (10%)	<code>docs/TESTING_STRATEGY.md</code> (section 3 of this document)	✓
README updated (5%)	badges, structure, how to run/test/release, CI/CD table, screenshots, changes to starter	✓
Coverage report (5%)	100% line coverage (minimum 80% enforced), HTML report artifact, <code>docs/COVERAGE.md</code> (Fig. 7)	✓

2. Screenshots of passing CI workflows

Platform Solutions Resources Open Source Enterprise Pricing

Search

Sign in Sign up

thongvx / ddm501-lab3-testing-cicd Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

main 2 Branches 1 Tag

Go to file

<> Code

thongvx ci: pin runner image to ubuntu-24.04 for reproducible builds 6 minutes ago 2 Commits

.github/workflows

ci: pin runner image to ubuntu-24.04 for reproducible builds 6 minutes ago

app

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

docs

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

models

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

scripts

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

tests

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

.dockerignore

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

.flake8

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

.gitignore

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

.pre-commit-config.yaml

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

Dockerfile

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

Makefile

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

README.md

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

pyproject.toml

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

requirements-dev.txt

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

requirements.txt

Lab 3: complete test suite, CI/CD pipelines, pre-commit an... 12 minutes ago

README

Lab 3: Testing & CI/CD for ML Systems

CI Pipeline passing CD Pipeline passing Model Validation passing coverage 100% tests 162 passed python 3.10 | 3.11
pre-commit enabled code style black

Comprehensive testing and CI/CD for the movie rating prediction API (FastAPI + Surprise SVD on MovieLens 100K).

Course: DDM501 – AI in Production: From Models to Systems · Lab 3 (15%) Student: Vương Xuân Thông – 25MS13306

Deliverable	Where
Test suite (unit, integration, data, model) – 162 tests	tests/
CI/CD pipelines (GitHub Actions)	ci.yml , cd.yml , model-validation.yml
Code quality setup	.pre-commit-config.yaml , .flake8 , pyproject.toml
Coverage report (100%, gate ≥ 80%)	docs/COVERAGE.md · HTML report = CI artifact coverage-report
Testing strategy document	docs/TESTING_STRATEGY.md

About

DDM501 Lab 3 – Testing & CI/CD for ML systems: movie rating API (FastAPI + SVD), 162 tests, 100% coverage, GitHub Actions CI/CD

Readme

Activity

0 stars

0 watching

0 forks

Report repository

Releases 1

Release v1.0.0 Latest 1 minute ago

Contributors 1

thongvx ThongVX

Languages

Python 95.6% Dockerfile 2.7% Makefile 1.7%

Figure 1. Repository with passing CI / CD / Model Validation badges

Platform
Solutions
Resources
Open Source
Enterprise
Pricing

thongvx/ddm501-lab3-testing-cicd
Public

<> Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights

Actions

All workflows
CD Pipeline
CI Pipeline
Model Validation
Management
Caches
Deployments

All workflows

9 workflow runs

Workflow
Event
Status
Branch
Actor

✓
ci: pin runner image to ubuntu-24.04 for reproducible builds
v1.0.0
3 minutes ago
3m 1s

CD Pipeline #1: Commit f64724e pushed by thongvx

✓
ci: pin runner image to ubuntu-24.04 for reproducible builds
v1.0.0
3 minutes ago
37s

Model Validation #4: Commit f64724e pushed by thongvx

✓
Model Validation
main
6 minutes ago
41s

Model Validation #3: Manually run by thongvx

✓
ci: pin runner image to ubuntu-24.04 for reproducible builds
develop
6 minutes ago
1m 41s

CI Pipeline #4: Commit f64724e pushed by thongvx

✓
ci: pin runner image to ubuntu-24.04 for reproducible builds
develop
6 minutes ago
39s

Model Validation #2: Commit f64724e pushed by thongvx

✓
ci: pin runner image to ubuntu-24.04 for reproducible builds
main
6 minutes ago
58s

Model Validation #1: Commit f64724e pushed by thongvx

✓
ci: pin runner image to ubuntu-24.04 for reproducible builds
main
6 minutes ago
1m 50s

CI Pipeline #3: Commit f64724e pushed by thongvx

✓
Lab 3: complete test suite, CI/CD pipelines, pre-commit and t...
develop
12 minutes ago
5m 47s

CI Pipeline #2: Commit 2230152 pushed by thongvx

✓
Lab 3: complete test suite, CI/CD pipelines, pre-commit and t...
main
12 minutes ago
5m 24s

CI Pipeline #1: Commit 2230152 pushed by thongvx

Figure 2. All workflow runs green (main, develop, tag v1.0.0)

PlatformSolutionsResourcesOpen SourceEnterprisePricing

SearchSign inSign up

thongvx/ddm501-lab3-testing-cicdPublic

NotificationsFork 0Star 0

<> CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

< CI Pipeline

ci: pin runner image to ubuntu-24.04 for reproducible builds #3Sign in to view logs

Summary

All jobs

Lint (pre-commit)

Type Check (mypy)

Tests (Python 3.10)

Tests (Python 3.11)

Build & Smoke-test Docker Image

Run details

Usage

Workflow file

Triggered via push 6 minutes ago

thongvx pushed f64724e main

Status

Success

Total duration

1m 50s

Artifacts

2

ci.yml

on: push

Lint (pre-commit)18s

Type Check (mypy)36s

Matrix: test

2 jobs completed

Show all jobs

Build & Smoke-test Docker Image37s

Artifacts

Produced during runtime

Name	Size	Digest
coverage-report	42.9 KB	sha256:a930d9a6470a410a8e045d69656d70d6873... copy
thongvx-ddm501-lab3-testing-cicd-EFUSS2.dockerbuild	33.3 KB	sha256:c697f35c013da749c96fada8c64951dbe14... copy

Figure 3. CI Pipeline: lint, type check, tests on Python 3.10 & 3.11, Docker smoke test

Platform

Solutions

Resources

Open Source

Enterprise

Pricing

Search

Sign in

Sign up

thongvx/ddm501-lab3-testing-cicd

Public

Notifications

Fork 0

Star 0

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

← CD Pipeline

ci: pin runner image to ubuntu-24.04 for reproducible builds #1

Sign in to view logs

Summary

All jobs

CI gate

Build and Push Docker Image

Deploy to Staging

Create GitHub Release

Deploy to Production

Run details

Usage

Workflow file

Triggered via push 4 minutes ago

thongvx pushed → f64724e v1.0.0

Success

Total duration 3m 1s

Artifacts 3

cd.yml

on: push

CI gate / Link (pre-commit)

2 jobs completed

C. / Build & Smoke test D...

Build and Push Docker Im...

Deploy to Staging

Create GitHub Release

Deploy to Production

Artifacts

Produced during runtime

Name	Size	Digest
coverage-report	42.9 KB	sha256:3c818bda81f60c0ee27de61f02b52679bac...
thongvx-ddm501-lab3-testing-cicd-JBTDLI.dockerbuild	67.2 KB	sha256:9f3dee4019ffbc05e7d98892154844b4233...
thongvx-ddm501-lab3-testing-cicd-MH8B5J.dockerbuild	32.6 KB	sha256:34bab639e5c0975443bdf3e7b91aad06bf...

Figure 4. CD Pipeline on tag v1.0.0: CI gate, build & push, staging, release, production

PlatformSolutionsResourcesOpen SourceEnterprisePricing

SearchSign inSign up

thongvx/ddm501-lab3-testing-cicdPublic

NotificationsFork 0Star 0

<> CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

Model Validation

Model Validation #3

Sign in to view logs

Summary

All jobs

Train & Validate Model

Run details

Usage

Workflow file

Manually triggered 6 minutes ago

Status

Total duration

Artifacts

thongvx f64724e main

Success

41s

1

model-validation.yml

on: workflow_dispatch

Train & Validate Model

36s

Artifacts

Produced during runtime

Name	Size	Digest
svd-model-f64724ea13b249cb48c753a450a95cf989686804	2.46 MB	sha256:4b6d8de7f1d0166af46e9cd6634379e3056...

Figure 5. Model Validation: training + RMSE/MAE quality gate

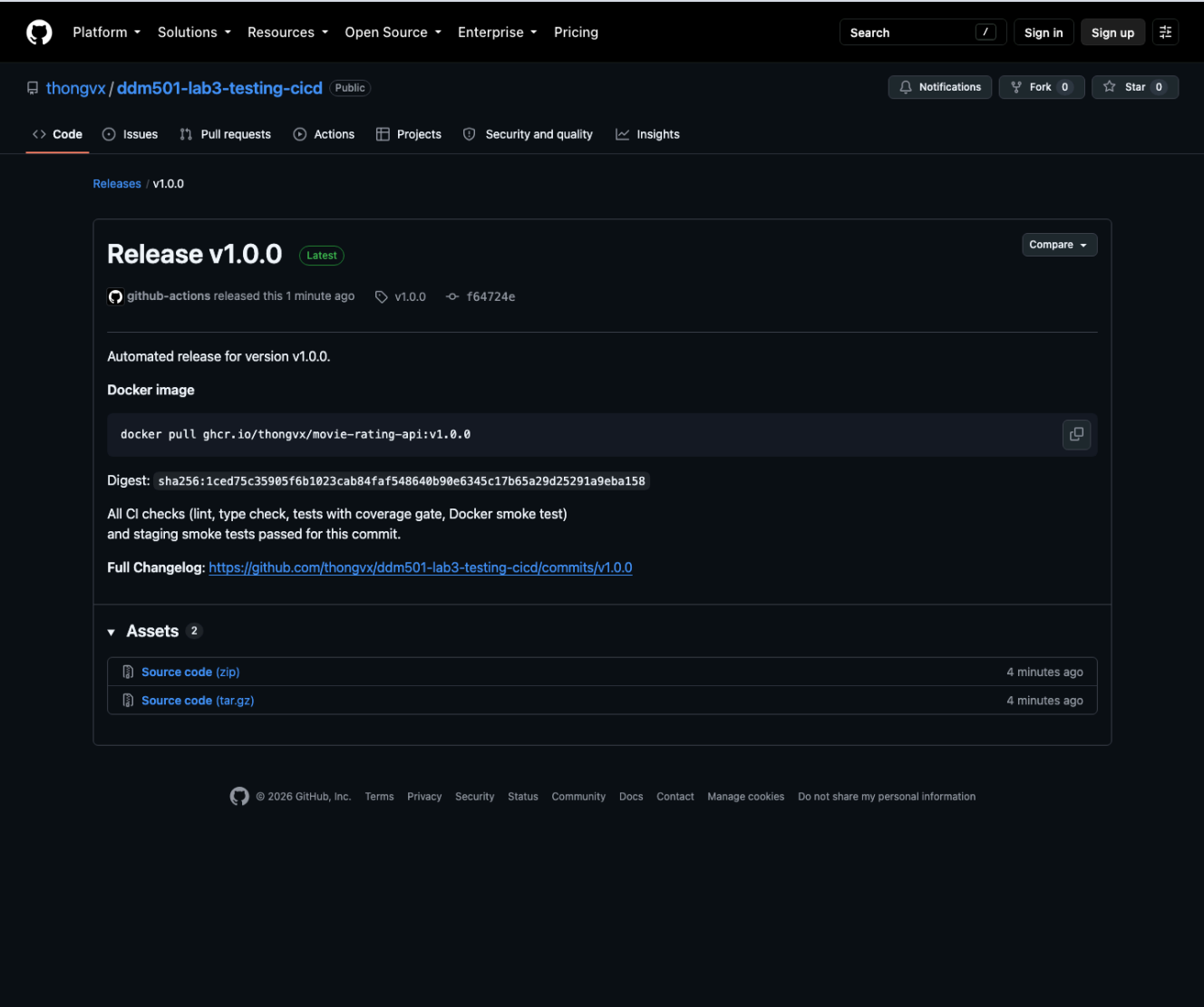


Figure 6. GitHub Release v1.0.0 created by the CD pipeline

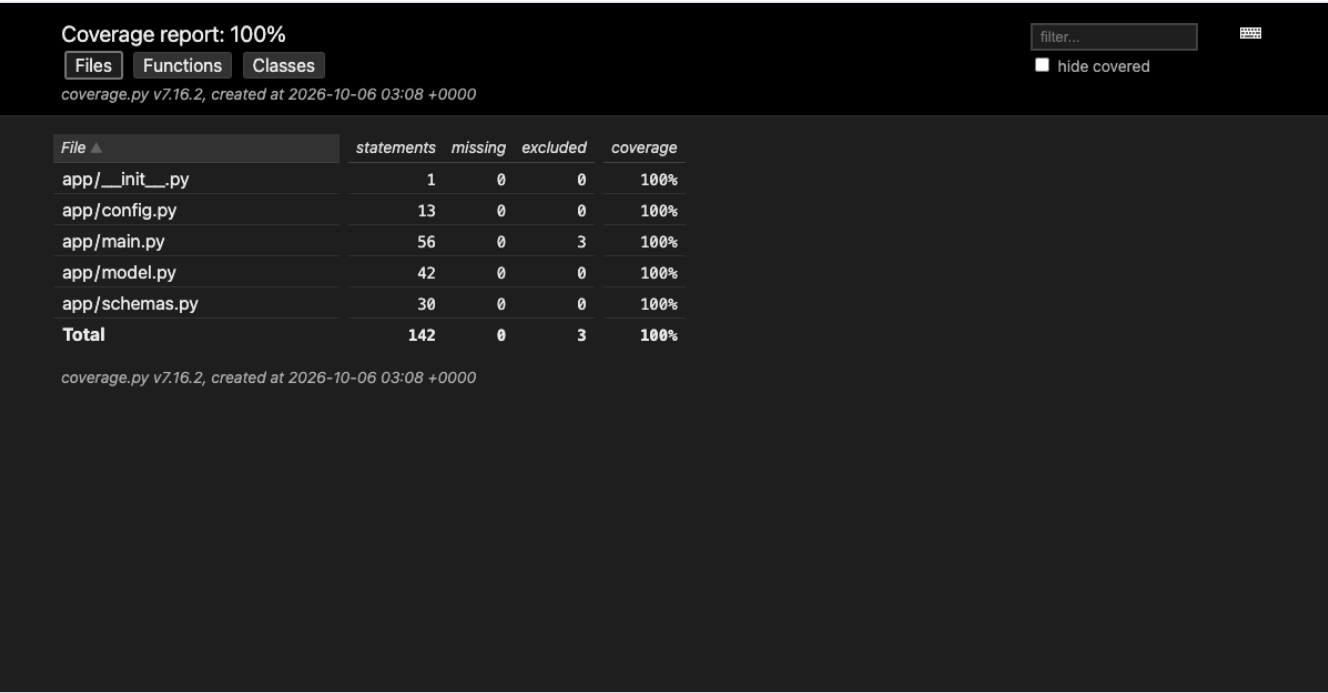


Figure 7. HTML coverage report (CI artifact coverage-report): 100%

3. Testing strategy

Testing Strategy – Movie Rating Prediction API

Course: DDM501 – AI in Production · **Lab 3:** Testing & CI/CD for ML Systems **Author:** Vương Xuân Thông (25MS13306)

This document explains *what* we test, *why*, *how* the tests are organised, and how the CI/CD pipeline uses them as quality gates before code or a model reaches production.

1. System under test

Component	File	Responsibility
Model wrapper	app/model.py	Load the pickled Surprise SVD model, predict one pair or a batch, clip to 1–5
API	app/main.py	FastAPI app: /, /health, /predict, /predict/batch, /model/info; model loaded in lifespan
Schemas	app/schemas.py	Pydantic v2 request/response contracts (ID length 1–50, rating 1–5, batch 1–100)
Config	app/config.py	Paths, versions and server settings, overridable via environment variables
Training	scripts/train_model.py	MovieLens 100K → SVD (seed 42); writes models/svd_model.pkl + hold-out metrics.json
Model gate	scripts/validate_model.py	Fails if the model does not load, predicts outside 1–5, or misses RMSE/MAE thresholds

2. Quality goals and risks

Risk in an ML service	Example failure	Guarded by
Code regressions	refactor breaks clipping or batch order	unit tests
Broken API contract	field renamed, wrong status code, internals leaked in errors	integration tests
Bad training data	duplicated ratings, ratings outside 1–5, missing IDs	data tests
Silently degraded model	model predicts the mean for everyone; worse than baseline	behavioural tests + model gate
Environment drift	dependency cannot be installed, image does not start	CI on Python 3.10/3.11, Docker smoke test
Unsafe release	untested tag shipped to production	CD re-runs CI, staging smoke test before release

3. Test pyramid

▲ System / E2E	Docker smoke tests (CI) + staging smoke tests (CD)
▲▲▲ Model behaviour performance, robustness	23 tests – invariance, directional, MFT,
▲▲▲▲ Data quality uniqueness, distribution	32 tests – schema, ranges, completeness,
▲▲▲▲▲ Integration latency	44 tests – every endpoint, validation, error paths,
▲▲▲▲▲▲▲ Unit	63 tests – model wrapper, schemas, config, helpers

Total: 162 tests, 100% line coverage of app/ (gate: $\geq 80\%$). The full suite runs in about 2 s once the model is trained, so there is no reason to skip it locally.

3.1 Unit tests – tests/unit/ (63)

File	Classes	What is verified
test_model.py	TestMovieRatingModel (14)	load, return type float , 2-decimal rounding, 1–5 range, batch type/length/range/empty, is_loaded , None /empty IDs
	TestModelWithStub (7)	isolated tests with a stub model pickled to tmp_path : clipping above 5 and below 1, rounding, order, RuntimeError when unloaded, TypeError for non-string IDs
	TestModelFileHandling (3), TestModelSingleton (1)	missing / corrupt file, stored path, get_model() singleton and reset_model()
test_schemas.py	5 classes (32)	required fields, empty / whitespace / None , stripping, 50-char limit and boundary, no int→str coercion, rating bounds 1.0/5.0, batch 1–100 and boundaries, nested validation
test_utils.py	TestConfig (4), TestMainHelpers (2)	env-var overrides, defaults, 503 helper, startup failure leaves API alive but unhealthy

The stub-model tests make the unit layer independent of training and cover branches the real model never reaches (e.g. a raw estimate of 7.3 must be clipped to 5.0).

3.2 Integration tests – tests/integration/test_api.py (44)

The FastAPI app is exercised through TestClient **inside a** with **block**, so the lifespan handler loads the real model exactly as in production (the starter's bare TestClient(app) skipped startup and every /predict returned 503).

- Happy paths for all 5 endpoints, response structure and echoes, model_version , OpenAPI schema.
- API result equals a direct model call (no hidden transformation); batch equals single predictions; batch preserves order.
- Validation: missing fields, empty body, invalid JSON, integer IDs, every invalid fixture, 10 000-char IDs, empty / >100 batches → **422** with the offending field in detail .
- Error handling: 404, 405 (GET /predict , POST /health), **503** when the model is missing (also /health → unhealthy), **500** when prediction raises – and the internal exception text is *not* leaked to the client. CORS header present.
- Non-functional: p95 latency of /predict through the HTTP stack < 50 ms.

3.3 Data tests – tests/data/test_data_quality.py (32)

Two data sources:

1. `sample_ratings` (fixture) – the lab's required checks: range, negatives, maximum, missing IDs, ID types, nulls/NaN, required fields, mean 2.0–4.5, $0 < \text{std} < 2.0$, distinct values, unique (user, movie) pairs, multiple users/movies, numeric types.
2. **The real MovieLens 100K file the model is trained on** – a data contract: exactly 100 000 rows, schema, ratings $\in \{1,2,3,4,5\}$, no missing IDs, no duplicate pairs, 943 users / 1 682 movies, every user ≥ 20 ratings, mean 3.0–4.0, std 0.8–1.4, no star value $> 50\%$, timestamps inside the collection period, and the "known pairs" used by model tests really exist with those ratings.

`TestDataValidationCatchesBadData` feeds corrupted records to the rules to prove the checks can fail (a test that can never fail protects nothing).

3.4 Model behavioural tests – `tests/model/test_model_behavior.py` (23)

Following the CheckList methodology:

Type	Tests
Invariance	same input $\rightarrow$ same output ($\times 2$, $\times 5$); batch order does not change per-pair results; batch = individual; a pair's score does not depend on its batch neighbours
Directional	different users / movies $\rightarrow$ different predictions; for > 100 users who gave both 5★ and 1★, the 5★ movie is predicted higher in $\geq 95\%$ of cases; for a new user, acclaimed movies (A Close Shave, Schindler's List, The Wrong Trousers) score above poorly rated ones
Minimum functionality	known users predicted in range; predictions not all identical; unknown users / movies get a valid fallback; unknown user + movie $\rightarrow$ global mean (≈ 3.53)
Performance	MAE < 1.0 on 1 000 real ratings; no error > 3.0 on known pairs; hold-out RMSE < 0.95 and MAE < 0.75 ; better than the global-mean baseline
Robustness	numeric strings, leading zeros ("001" is a different, unknown user – documented), Unicode and 500-char IDs

Why " $\geq 80\%$ within 1.5 stars" instead of "every pair within 1.5": the starter suggested every known pair should be within 1.5 stars. User 166 rated movie 346 one star, but SVD predicts ≈ 2.8 because it regularises towards user and item means. Loosening the tolerance until it passes would hide information, so the test asserts what a good collaborative-filtering model can actually guarantee (most pairs close, none absurd) and the real accuracy requirement is moved to a proper hold-out set (`test_holdout_metrics_meet_thresholds`).

3.5 System / end-to-end tests (in the pipelines)

- **CI docker job:** builds the multi-stage image, runs it, checks `/health` reports `model_loaded: true`, a valid `/predict`, a 422 for a bad request, and waits for Docker's own HEALTHCHECK to report `healthy`.
- **CD deploy-staging job:** pulls the image that was just pushed (by digest), runs it and smoke-tests `/health`, `/predict`, `/predict/batch`, `/model/info` before a release is created.

4. Fixtures and test data

All shared fixtures live in `tests/conftest.py`:

Fixture	Scope	Purpose
<code>test_client</code>	session	<code>TestClient</code> with lifespan $\rightarrow$ real model loaded once
<code>trained_model</code>	session	<code>MovieRatingModel()</code> ; skips with a clear message if not trained

Fixture	Scope	Purpose
<code>model_metrics</code>	session	hold-out metrics from <code>models/metrics.json</code>
<code>movielens_ratings</code>	session	the 100 000 real ratings (skips if the dataset is not downloaded)
<code>sample_*</code> , <code>invalid_prediction_requests</code> , <code>known_user_movie_pairs</code> , <code>unknown_users/movies</code>	function	small deterministic inputs

Determinism: the model is trained with `random_state=42`, samples use `random.Random(seed)`, so the suite gives identical results on every run and platform.

5. Quality gates in CI/CD

```

push / PR → Lint (pre-commit: black, isort, flake8, mypy, hygiene)
           → Type check (mypy, disallow-untyped-defs)
                                     ↓
           Tests on Python 3.10 & 3.11: train ► model gate ► 162 tests ► coverage ≥ 80%
                                     ↓
           Docker: build multi-stage image ► smoke test ► HEALTHCHECK healthy

tag v* → CI (reused) ► build & push GHCR ► staging smoke test ► GitHub Release ► production

model/script/dependency change, weekly → Model Validation: 5-fold CV ► RMSE ≤ 0.95, MAE ≤ 0.75

```

Gate	Threshold	Where
Formatting / lint	black, isort, flake8 clean	pre-commit hook + CI lint
Types	mypy, no untyped defs in <code>app/</code> , <code>scripts/</code>	pre-commit + CI type-check
Tests	162/162 pass on 3.10 and 3.11	CI test
Coverage	≥ 80% (<code>--cov-fail-under=80</code> , <code>fail_under</code> in <code>pyproject.toml</code>)	CI test
Model quality	hold-out RMSE ≤ 0.95, MAE ≤ 0.75	CI test, Model Validation workflow
Runtime	container healthy, endpoints respond	CI docker, CD deploy-staging

6. Running the tests

```

pip install -r requirements.txt -r requirements-dev.txt
python scripts/train_model.py          # once: downloads MovieLens 100K, trains, writes metrics
pytest tests/ --cov=app --cov-report=html # full suite + HTML report in htmlcov/
pytest tests/unit/ -v                  # one layer
pytest -m integration                  # by marker
pre-commit run --all-files              # everything CI's lint job runs

```

7. Policies

- **New code ships with tests;** coverage may not drop below 80% (the build fails).

- **Flaky tests are bugs:** tests use fixed seeds and no wall-clock assumptions except the generous 50 ms latency check; a flaky test is fixed or quarantined with an issue, never retried silently.
- **Tests must be able to fail:** every rule-based check has a negative test.
- **Model changes** go through the Model Validation workflow; thresholds change only in a reviewed PR.

8. Limitations and next steps

- Load testing (Locust/k6) against the container to validate throughput, not just single-request latency.
- Mutation testing (`mutmut`) to measure test strength beyond line coverage.
- Fairness/slice tests (e.g. error by user activity level) and drift tests on new rating logs.
- Contract tests for API consumers (schemathesis on the OpenAPI schema).

4. Coverage report

Test Coverage Report

Line coverage of `app/` : **100%** (162 tests passed) · CI gate: `--cov-fail-under=80` and `fail_under = 80` in `pyproject.toml`.

Name	Stmts	Miss	Cover	Missing
<code>app/__init__.py</code>	1	0	100%	
<code>app/config.py</code>	13	0	100%	
<code>app/main.py</code>	56	0	100%	
<code>app/model.py</code>	42	0	100%	
<code>app/schemas.py</code>	30	0	100%	
TOTAL	142	0	100%	

Generated with:

```
python scripts/train_model.py --skip-cv
pytest tests/ --cov=app --cov-report=term-missing --cov-report=html --cov-fail-under=80
```

- Every CI run (Python 3.10 and 3.11) prints this table in the job summary and uploads the full **HTML report** (`htmlcov/`), `coverage.xml` and `junit.xml` as the artifact `coverage-report` (Actions → CI Pipeline → run → *Artifacts*).
- Tests per layer: unit 63 · integration 44 · data 32 · model behaviour 23.
- The only excluded line is `if __name__ == "__main__":` in `app/main.py` (local dev server entry point).